System Analysis Documentation (SAD)

UMHackathon 2026

Project: SkyRescue AI - Automated
Drone Rescue Version: 1.0

1. Introduction

1.1. Purpose

This system analysis document describes the technical scope and design decisions behind Rescue Swarm Simulation. The document is meant to help developers, testers, and reviewers understand how the rescue swarm is structured, how mission state moves through the system, and how the command agent, MCP layer, API bridge, and browser UI work together.

The key elements covered in this document are:

1.1 Architecture

- Data flows
- Command-agent and LLM integration
- Scope boundaries and non-functional requirements
- Monitoring, assumptions, external dependencies, and delivery planning

1.2. Background

- The product addresses a disaster-response scenario where infrastructure may collapse during the first hours of a major event.
- In that environment, a rescue system that depends on a cloud-first, centralized workflow can fail when connectivity disappears.
- This implementation uses a local-first simulation core to model an edge-oriented rescue swarm. The command path is split across a browser UI, FastAPI bridge, MCP tools, and an orchestrator that can use a structured LLM plan when available or fall back to heuristic logic when the model is unavailable.

1.3. Role of Reference

This document is a reference for developers, QA, and technical reviewers. It provides a high-level map of the simulation engine, the API surface, the MCP surface, the browser client, and the planning loop so the team can reason about behaviour without reading every source file first.

1.4. Target Stakeholder

Stakeholder	Role	Expectations
Operator	Starts and monitors missions from the browser UI	Clear mission status, safe drone control, visible coverage and survivor progress
Command Agent / Orchestrator	Converts mission goals into tool calls	Accurate world state, valid fleet discovery, safe assignments, fallback when LLM is unavailable
Developer	Maintains the engine, API, MCP tools, and UI	Modular code, clear contracts, easy debugging, and minimal duplicated state
QA Team	Validates mission flows and boundary cases	Repeatable scenarios, observable logs, and graceful failure handling
Hackathon Judge	Reviews the demo and the solution quality	Clear architecture story, working rescue flow, and robust fallback behavior

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Browser-based rescue swarm simulation
Architecture	Local-first modular system with a shared in-memory simulation core, FastAPI bridge, FastMCP server, and an optional LLM-assisted orchestrator. The system is not deployed as separate cloud microservices. Instead, the SimulationEngine acts as the single source of truth, and both the REST API and the MCP server read and write to that same engine instance. The browser UI visualizes the world, issues mission commands, and receives live state updates. The orchestrator pulls world snapshots, optionally asks an LLM for a structured plan, and sends the plan back through MCP for assignment.

2.1.2. LLM as a Service Layer

The LLM is treated as a planning service rather than a generic AI module.

- The orchestrator fetches a compact world snapshot through MCP.

- It builds a structured prompt with mission status, hazard summary, survivor counts, wind, and fleet state.
- If an Anthropic-compatible API key is available, the orchestrator calls the model and expects JSON output only.
- The returned JSON is parsed into a plan and pushed back into the MCP layer with `set_plan`.
- If the model is missing, invalid, or returns unusable content, the system falls back to a heuristic rescue-first plan.

This design keeps the mission operational even when cloud access is unavailable. It also keeps token usage small because the orchestrator does not forward the full raw grid or the full mission log into the model context.

2.1.3. Dependency Diagram

System Flow

1. Browser UI (Three.js) -> sends requests to -> FastAPI Bridge
2. FastAPI Bridge -> interacts with -> SimulationEngine (Singleton)
3. Orchestrator -> communicates with -> MCP Server
4. MCP Server -> reads/writes to -> SimulationEngine
5. Orchestrator -> queries -> Anthropic-compatible LLM
6. Orchestrator -> sends plans via -> MCP Server
7. SimulationEngine -> updates -> Browser UI (Three.js)
8. SimulationEngine -> writes to -> Engine + Mission Logs
9. MCP Server -> writes to -> Engine + Mission Logs
10. Orchestrator -> writes to -> Engine + Mission Logs
11. Swarm System -> interacts with -> SimulationEngine
12. MCP Server -> coordinates with -> Swarm System

Key points

- The UI talks to the API bridge for mission control and telemetry.
- The orchestrator talks to the MCP server for world discovery and command dispatch.
- The engine is shared by every control path, so state does not fork.
- Logging is written to files and to the mission log shown in the UI.

2.2. Sequence Diagram

2.2.1. Model Process

The core mission loop is:

1. Operator starts a mission.
2. The engine seeds hazards and survivors and resets the world.
3. The orchestrator polls the world through MCP.
4. The LLM, if enabled, returns a compact plan.
5. The swarm system assigns drones to targets using battery-safe logic.
6. Drones move, scan, and report telemetry.
7. The engine updates coverage, survivor discovery, and logs.

2.2.2. Sequence Diagram

System Execution Flow (End-to-End Interaction)

1. Operator -> starts mission via -> Browser UI
2. Browser UI -> sends POST /commands/start -> FastAPI Bridge
3. FastAPI Bridge -> calls start_mission() -> SimulationEngine
4. SimulationEngine -> returns mission started -> FastAPI Bridge
5. FastAPI Bridge -> returns success -> Browser UI
6. Orchestrator -> requests world state -> MCP Server
7. MCP Server -> calls get_world_state() -> SimulationEngine
8. SimulationEngine -> returns world snapshot -> MCP Server
9. MCP Server -> returns snapshot -> Orchestrator
10. Orchestrator -> sends structured prompt (optional) -> LLM
11. LLM -> returns JSON plan or failure -> Orchestrator
12. Orchestrator -> sends plan -> MCP Server
13. MCP Server -> triggers swarm assignment flow -> SimulationEngine
14. Browser UI -> reports telemetry / thermal scan -> FastAPI Bridge
15. FastAPI Bridge -> calls update_drone_telemetry() / thermal_scan() -> SimulationEngine
16. SimulationEngine -> returns updated state -> Browser UI

2.3 Technological Stack

2.3.1. Frontend

- HTML, CSS, and JavaScript
- Three.js for the 3D mission scene
- HUD, minimap, logs, and settings controls in the browser

2.3.2. Backend

- Python 3.12
- FastAPI for HTTP and WebSocket endpoints
- Uvicorn for local serving
- FastMCP for the MCP tool surface

2.3.3. Agent and Planning Layer

- Python orchestrator in `agents/orchestrator.py`
- AnyIO for async MCP calls
- Anthropic-compatible SDK for optional structured planning

2.3.4. Storage and State

- In-memory singleton `SimulationEngine`
- File-based logs in `logs/`
- No persistent relational database in the current MVP

2.3.5. Deployment

- Local machine or demo server
- Browser loads the UI and reads live mission state from the API bridge
- No physical drone hardware required

2.4. Key Data Flows

2.4.1. Main Data Flow Narrative

- Mission start updates the engine, which generates hazards, survivors, and drone state.
- The UI reads the world state through the API and shows mission coverage, discovery, and battery.
- The orchestrator reads the world through MCP, creates a compact prompt, and optionally calls the LLM.
- The resulting plan is stored in MCP and interpreted by the swarm system.
- The swarm system assigns targets by checking battery safety, duplicate suppression, and target ownership.
- Telemetry from the UI is pushed back to the engine so movement and scanning remain synchronized.
- Thermal scans and hazard scans update sector state and survivor discovery, then the UI reflects the new state.

2.4.2. In-Memory State Model

Because the MVP is simulation-only, persistence is intentionally simple. There is no SQL database and no distributed message bus. The engine keeps the authoritative state in memory and writes human-readable logs to disk.

This keeps the mission responsive, but it also means

- Restarting the process resets the world
- Mission history is not durable across restarts
- The UI, API, and MCP tools must always read from the same engine instance

3. Functional Requirements & Scope

No.	Feature	Description
1	Mission Lifecycle Control	Start, reset, pause, and resume the mission
2	Autonomous Command Agent	Turn a high-level rescue goal into a sequence of tool calls
3	MCP Fleet Discovery and Tooling	Discover active drones and control them with standardized tools
4	Hazard and Survivor Detection	Scan sectors, detect fire/smoke, and identify survivors
5	Battery-Safe Resource Management	Prevent unsafe assignments and recall drones before failure
6	UI Visibility	Show 3D world state, minimap, HUD, and mission log

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	Mission commands should not corrupt shared state	Use one SimulationEngine instance and explicit state transitions
Offline resilience	Mission should keep running when the LLM is absent	Heuristic fallback plan and battery-safe swarm logic
Maintainability	Core behavior should be easy to reason about	Modular files for engine, swarm, MCP, orchestrator, UI, and logger
Responsiveness	UI and API should feel interactive in a demo setting	Lightweight in-memory state and compact prompt construction
Observability	Operators need to see what the system is doing	Mission log, file logs, drone logs, and live world state
Cost efficiency	LLM usage should stay controlled	Rate limiting and summarized prompt context

Testability	Scenarios should be repeatable	Fixed survivor counts, deterministic seeds for QA, and isolated mission resets

3.3. Out of Scope / Future Enhancements

- Real drone hardware integration
- GPS or map tile services
- Persistent database and mission replay store
- Multi-user collaborative control
- Live computer vision and camera feeds
- Production emergency dispatch integration
- Full rescue extraction and medical workflow simulation
- Peer-to-peer swarm networking between drones

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Seeded Scenario Rollout and Regression

Instead of a production grayscale rollout, the practical evaluation strategy for this project is scenario-based regression:

- Run a known mission seed and verify the baseline behavior
- Re-run the same seed after code changes and compare coverage, survivor discovery, and assignment behavior
- Compare heuristic fallback versus LLM-assisted planning on the same scenario

4.1.2. Emergency Rollback and Golden Release

If a regression breaks the mission flow, the system should revert to the latest known-good commit and known-good mission seed. The "golden release" for this project is the stable version where the mission start, assignment, scan, recall, and fallback flows all work without manual repair.

4.1.3. Priority Matrix

Priority	Trigger Condition	Action
P0 Critical	Mission crash, deadlock, or unsafe assignment that cannot be recovered	Stop demo, revert to golden build, investigate immediately
P1 High	LLM produces invalid JSON or swarm creates repeated oscillation	Enable fallback logic, fix parser or cooldown rules

P2 Medium	UI desync, stale telemetry, or slow state refresh	Investigate API and WebSocket timing
P3 Low	Cosmetic issues or non-blocking log noise	Track for later cleanup

4.2. Monitoring The system should be monitored through

- Engine log files in `logs/`
- Per-drone log files
- Mission log entries surfaced in the HUD
- API health checks through `/settings`, `/state`, and the command endpoints
- WebSocket connectivity for live world refresh
- LLM parse failure counts and heuristic fallback usage

4.3. Assumptions

- The project is simulation-only and does not require physical drone hardware
- The browser supports WebGL and ES module imports
- The mission runs in one local Python process in the current MVP
- The LLM is optional, not mandatory
- The active fleet can change during a mission, so discovery must happen at runtime
- Mission state is transient and is reset on restart unless persistence is added later

4.4. External Dependencies

Technology	Purpose	Risk / Notes
FastAPI	REST API and WebSocket bridge	Low risk, but the API must stay stable
Uvicorn	Local app server	Low risk, mainly operational
FastMCP	Standardized tool exposure for the command agent	Medium risk if tool contracts drift
Anthropic-compatible SDK	Optional structured planning model	High risk if the endpoint is unavailable or returns invalid output
AnyIO	Async bridge for MCP calls	Low risk, supports the orchestrator loop
Three.js	3D visualization in the browser	Medium risk if rendering fails in older browsers
Browser WebGL	Mission scene rendering	Medium risk on unsupported devices
httpx	HTTP client utility in the Python environment	Low risk

5. Project Management & Team Contributions

5.1. Project Timeline

The preliminary sprint is best treated as a short demo-focused delivery cycle. A practical 10-day plan is:

Day 1-2

- Confirm architecture and state model
- Finalize the mission flow and control surface

Day 3-4

- Harden SimulationEngine behaviour
- Verify hazard, survivor, and battery rules

Day 5-6

- Validate MCP tool contracts
- Verify orchestrator prompt construction and fallback logic

Day 7-8

- Polish the browser UI, mini-map, and mission log
- Verify telemetry and charging state transitions

Day 9

- Run scenario-based QA
- Fix regression issues

Day 10

- Final demo rehearsal and documentation pass

5.2. Team Members Role

Role	Responsibilities
Simulation / Backend Lead	Owns the engine, state transitions, hazards, battery model, and mission rules
MCP / Orchestrator Lead	Owns runtime fleet discovery, prompt construction, plan parsing, and assignment flow
Frontend / Visualization Lead	Owns the Three.js scene, HUD, minimap, telemetry display, and controls
QA / Documentation Lead	Owns test cases, validation flows, documentation, reporting, and overall system verification

5.3. Recommendations

- Add automated unit and integration tests for the engine, MCP server, and orchestrator
- Add a small CI workflow so the mission flow is exercised on every push
- Add deterministic mission seeds for repeatable demo and QA runs
- Add a compact mission summary endpoint or export so testers can compare runs more easily
- Consider a persistence layer only if mission replay or audit history becomes a required feature
- Keep the LLM optional and maintain a strong heuristic fallback so the demo still works offline